

ISYE 671: Network Flow Models

Lecture Notes — Week [X]

Instructor: Dr. Ziteng Wang | **Course:** ISYE 671 — Linear Optimization and Network Flows

Duration: 2 hours | **Prerequisites:** Linear Programming, Integer Programming

Learning Objectives

By the end of this session, students will be able to:

1. Define and use graph notation: nodes, arcs, directed/undirected graphs, paths, chains, and cycles
 2. Formulate the **minimum cost network flow** (MCNF) problem and recognize its LP structure
 3. Identify source, sink, and transshipment nodes, and understand flow conservation constraints
 4. Formulate **maximum flow** and **shortest path** problems as special cases of MCNF
 5. Formulate **assignment** and **matching** problems and recognize their network structure
-

1. Graph Fundamentals and Notation

1.1 Directed Graphs (Digraphs)

A **directed graph** (digraph) $G = (V, A)$ consists of:

- $V = \{1, 2, \dots, n\}$: a finite set of **nodes** (or vertices)
- $A \subseteq V \times V$: a set of **arcs** (directed edges), where each arc $(i, j) \in A$ points from node i to node j

We call i the **tail** and j the **head** of arc (i, j) .

1.2 Undirected Graphs

An **undirected graph** $G = (V, E)$ has **edges** instead of arcs. An edge $\{i, j\} \in E$ connects nodes i and j with no implied direction. Undirected graphs can always be represented as digraphs by replacing each edge $\{i, j\}$ with two opposing arcs (i, j) and (j, i) .

1.3 Key Graph Concepts

Term	Definition
Path	A sequence of arcs connecting two nodes, all traversed in the forward direction. No node is visited more than once.
Chain	Like a path, but arcs may be traversed against their direction.
Cycle	A chain that starts and ends at the same node.
Dicycle	A cycle in which all arcs are traversed in the forward direction.
Bipartite graph	A graph whose nodes can be partitioned into two disjoint sets S and T , with every arc/edge having one endpoint in each set.

1.4 Node–Arc Incidence Matrix

The structure of a digraph can be captured algebraically by its **node–arc incidence matrix** A , which has one row for every node and one column for every arc. For arc (i, j) :

- Entry in row i (tail): -1
- Entry in row j (head): $+1$
- All other entries: 0

Example (Rardin Example 10.5). Consider a 4-node digraph with arcs $(1, 2)$, $(1, 4)$, $(2, 4)$, $(3, 1)$, $(4, 3)$. The node–arc incidence matrix is:

Node	(1, 2)	(1, 4)	(2, 4)	(3, 1)	(4, 3)
1	-1	-1	0	$+1$	0
2	$+1$	0	-1	0	0
3	0	0	0	-1	$+1$
4	0	$+1$	$+1$	0	-1

Key property: Every column has exactly one $+1$ and one -1 . This special structure gives network flow problems their remarkable computational properties, including **total unimodularity** (Section 6 below).

Exercise 1. Given the following node–arc incidence matrix, draw the corresponding digraph.

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6
1	-1	0	0	$+1$	0	0
2	$+1$	-1	0	0	$+1$	0
3	0	$+1$	-1	0	0	0
4	0	0	$+1$	-1	0	-1
5	0	0	0	0	-1	$+1$

2. The Minimum Cost Network Flow Problem

2.1 Problem Statement

The **minimum cost network flow (MCNF)** problem is the foundational model for all network flow optimization. Given a digraph $G = (V, A)$ with:

- **Decision variables:** x_{ij} = amount of flow on arc (i, j)
- **Parameters for each arc** $(i, j) \in A$:
 - c_{ij} = unit cost of flow
 - u_{ij} = capacity (upper bound on flow)
- **Parameters for each node** $k \in V$:
 - b_k = **net demand** at node k

The MCNF formulation is:

$$\begin{aligned}
& \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\
& \text{s.t.} \quad \sum_{(i,k) \in A} x_{ik} - \sum_{(k,j) \in A} x_{kj} = b_k \quad \forall k \in V \\
& \quad \quad 0 \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A
\end{aligned}$$

2.2 Flow Conservation (Balance) Constraints

The main constraints enforce **conservation of flow** at every node:

$$(\text{total flow in}) - (\text{total flow out}) = \text{net demand } b_k$$

2.3 Node Types

Node Type	Net Demand b_k	Role
Source (supply)	$b_k < 0$	Creates/generates flow
Sink (demand)	$b_k > 0$	Consumes flow
Transshipment	$b_k = 0$	Passes flow through

Important: For the problem to be feasible, we require $\sum_{k \in V} b_k = 0$, i.e., total supply must equal total demand. If total supply exceeds total demand, a dummy sink node is added with zero-cost arcs from all sources to absorb the excess.

2.4 Matrix Form

In matrix notation, the MCNF becomes a standard LP:

$$\min \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} = \mathbf{b}, \quad \mathbf{0} \leq \mathbf{x} \leq \mathbf{u}$$

where A is the node–arc incidence matrix.

2.5 Application: Optimal Ovens Inc. (OOI) — Rardin Application 10.1

Setting: Optimal Ovens Inc. (OOI) manufactures ovens at 2 plants (Wisconsin and Alabama, each producing 1000 units) and ships through 2 warehouses (Fresno and Pittsburgh) to 3 customers (Memphis: 450, Peoria: 500, Newark: 610). The warehouses can transfer at most 25 ovens between themselves in either direction.

Network structure:

- Nodes: 1 (Wisconsin), 2 (Alabama), 3 (Fresno), 4 (Pittsburgh), 5 (Memphis), 6 (Peoria), 7 (Newark), 8 (Dummy sink for excess supply: 440)
- Sources: nodes 1 and 2 with $b_1 = b_2 = -1000$

- Transshipment: nodes 3, 4 with $b_3 = b_4 = 0$
- Sinks: node 5 ($b_5 = 450$), node 6 ($b_6 = 500$), node 7 ($b_7 = 610$), node 8 ($b_8 = 440$)
- Arcs carry unit shipping costs and capacities

Formulation:

$$\min 7x_{1,3} + 8x_{1,4} + 4x_{2,3} + 7x_{2,4} + 25x_{3,5} + 5x_{3,6} + 17x_{3,7} + 29x_{4,5} + 8x_{4,6} + 5x_{4,7}$$

subject to:

Constraint	Equation	Node
Supply	$-x_{1,3} - x_{1,4} - x_{1,8} = -1000$	Node 1 (Wisconsin)
Supply	$-x_{2,3} - x_{2,4} - x_{2,8} = -1000$	Node 2 (Alabama)
Transship	$x_{1,3} + x_{2,3} + x_{4,3} - x_{3,4} - x_{3,5} - x_{3,6} - x_{3,7} = 0$	Node 3 (Fresno)
Transship	$x_{1,4} + x_{2,4} + x_{3,4} - x_{4,3} - x_{4,5} - x_{4,6} - x_{4,7} = 0$	Node 4 (Pittsburgh)
Demand	$x_{3,5} + x_{4,5} = 450$	Node 5 (Memphis)
Demand	$x_{3,6} + x_{4,6} = 500$	Node 6 (Peoria)
Demand	$x_{3,7} + x_{4,7} = 610$	Node 7 (Newark)
Dummy	$x_{1,8} + x_{2,8} = 440$	Node 8 (Dummy)
Capacity	$x_{3,4} \leq 25, x_{4,3} \leq 25$	Warehouse transfers
Bounds	$x_{ij} \geq 0$ for all $(i, j) \in A$	

Discussion points:

- Why was node 8 added? Total supply (2000) > total demand (1560), so excess = 440
- The zero-cost arcs (1, 8) and (2, 8) let excess supply “disappear” without affecting costs
- Node-arc incidence matrix has 8 rows (nodes) and 14 columns (arcs)

2.6 Smaller Worked Example — Rardin Example 10.1

Consider a 4-node network with $b_1 = -100$, $b_2 = 0$ (transshipment), $b_3 = 60$, $b_4 = 40$:

Arc	Cost c_{ij}	Capacity u_{ij}
(1, 2)	2	90
(1, 4)	3	75
(2, 3)	5	50
(2, 4)	0	∞
(4, 2)	-1	∞
(4, 3)	11	∞

Formulation:

$$\min 2x_{1,2} + 3x_{1,4} + 5x_{2,3} - x_{4,2} + 11x_{4,3}$$

$$\begin{aligned}
\text{s.t.} \quad & -x_{1,2} - x_{1,4} = -100 \\
& x_{1,2} + x_{4,2} - x_{2,3} - x_{2,4} = 0 \\
& x_{2,3} + x_{4,3} = 60 \\
& x_{1,4} + x_{2,4} - x_{4,2} - x_{4,3} = 40 \\
& x_{1,2} \leq 90, \ x_{1,4} \leq 75, \ x_{2,3} \leq 50; \quad x_{ij} \geq 0 \quad \forall (i,j) \in A
\end{aligned}$$

Exercise 2. A company has 3 factories (F_1, F_2, F_3) with supplies of 200, 150, and 100 units respectively, and 2 distribution centers (D_1, D_2) serving 2 retail stores (R_1 : 180 demand, R_2 : 250 demand). Shipping costs per unit are: $F_1 \rightarrow D_1$: \$3, $F_1 \rightarrow D_2$: \$5, $F_2 \rightarrow D_1$: \$4, $F_2 \rightarrow D_2$: \$2, $F_3 \rightarrow D_2$: \$6, $D_1 \rightarrow R_1$: \$2, $D_1 \rightarrow R_2$: \$4, $D_2 \rightarrow R_1$: \$3, $D_2 \rightarrow R_2$: \$1. Draw the network, identify node types, add a dummy sink if needed, and write the full MCNF formulation.

3. Maximum Flow Problem as a Special Case of MCNF

3.1 Problem Definition

Given a digraph $G = (V, A)$ with a designated **source** s , **sink** t , and arc capacities u_{ij} , the **maximum flow problem** seeks the largest total flow from s to t subject to flow conservation and capacity constraints.

3.2 Conversion to MCNF

The max flow problem is a special case of MCNF. The conversion works by:

1. Set all arc costs $c_{ij} = 0$ on the original arcs
2. Set all node demands $b_k = 0$ for all $k \in V$
3. Add a **return arc** (t, s) from sink to source with cost $c_{t,s} = -1$ and unlimited capacity

Minimizing the MCNF objective (which is $-1 \times x_{t,s}$) is equivalent to maximizing $x_{t,s}$, the total flow from s to t .

MCNF formulation for max flow:

$$\begin{aligned}
& \min \quad -x_{t,s} \\
\text{s.t.} \quad & \sum_{(i,k) \in A'} x_{ik} - \sum_{(k,j) \in A'} x_{kj} = 0 \quad \forall k \in V \\
& 0 \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A, \quad x_{t,s} \geq 0
\end{aligned}$$

where $A' = A \cup \{(t, s)\}$.

3.3 Max Flow – Min Cut Duality (*Optional*)

One of the most celebrated results in combinatorial optimization:

Max Flow – Min Cut Theorem (Ford & Fulkerson, 1956): The maximum total feasible flow from s to t equals the minimum total forward capacity of any cut separating s and t .

An s - t **cut** (S, T) partitions V into S (containing s) and $T = V \setminus S$ (containing t). Its **capacity** is $\sum_{i \in S, j \in T, (i,j) \in A} u_{ij}$.

3.4 Application: Building Evacuation — Rardin Application 10.5

Setting: A proposed sports arena must be evaluated for emergency evacuation capacity. Patrons exit through 4 doors (capacity 600/min each) into an outer hallway (350/min in each direction), then through 4 firestairs (400/min each) and a parking tunnel (800/min) to safety.

The network has source $s = 1$ (arena), sink $t = 10$ (outside), and intermediate nodes 2–9 representing hallway junctions, stairways, and the tunnel entrance.

Result: Maximum evacuation rate = **2,100 persons per minute**.

Minimum cut identifies the bottleneck: $S = \{1, 4, 5, 6, 7, 8\}$, $T = \{2, 3, 9, 10\}$ with forward arcs across the cut having total capacity $600 + 350 + 350 + 400 + 400 = 2,100$, matching the max flow.

3.5 Worked Example — Rardin Example 10.24

Consider the following max flow instance with arc labels (u_{ij}, x_{ij}) denoting capacity and current flow:

Arc	Capacity u_{ij}	Current Flow x_{ij}
(1, 2)	40	15
(1, 3)	10	10
(2, 3)	13	3
(2, 4)	12	10
(3, 4)	22	15

Maximum flow = 30, achieved at $x_{1,2} = 20$, $x_{1,3} = 10$, $x_{2,3} = 8$, $x_{2,4} = 12$, $x_{3,4} = 18$.

Minimum cut separates $\{1, 2\}$ from $\{3, 4\}$ with forward capacity $10 + 8 + 12 = 30$.

Augmenting paths from the initial flow:

- Path $1 \rightarrow 2 \rightarrow 4$: both arcs forward, $\lambda = \min\{25, 2\} = 2$
- Path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$: arc (1, 2) forward, arc (3, 2) reverse, arc (3, 4) forward; $\lambda = \min\{25, 3, 7\} = 3$

Exercise 3. A city water system has source s and sink t connected through 4 intermediate pumping stations. Arc capacities (in million gallons/day) are: $(s, 1)$: 15, $(s, 2)$: 12, $(1, 3)$: 8, $(1, 2)$: 5, $(2, 4)$: 10, $(3, t)$: 12, $(3, 4)$: 3, $(4, t)$: 14. Find the maximum flow and identify a minimum cut. Then: if you could increase the capacity of exactly one arc by 5 units, which would you choose and why?

4. Shortest Path Problem as a Special Case of MCNF

4.1 Problem Definition

Given a digraph $G = (V, A)$ with arc costs c_{ij} and a designated source node s and destination node t , the **shortest path problem** finds the minimum-cost path from s to t .

4.2 Conversion to MCNF

The shortest path problem is a special case of MCNF with:

- **One unit of flow** to send from s to t
- Net demands: $b_s = -1$ (source), $b_t = +1$ (sink), $b_k = 0$ for all other nodes
- All capacities $u_{ij} = \infty$ (or 1 suffices for a single-path solution)
- Arc costs c_{ij} represent distances, times, or other costs

MCNF formulation:

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{(i,k) \in A} x_{ik} - \sum_{(k,j) \in A} x_{kj} = b_k \quad \forall k \in V \\ & x_{ij} \geq 0 \quad \forall (i,j) \in A \end{aligned}$$

where $b_s = -1$, $b_t = +1$, and $b_k = 0$ otherwise.

At optimality, arcs with $x_{ij} = 1$ form the shortest path.

Exercise 4. Formulate the shortest path from node 1 to node 6 in the following network as an MCNF. Nodes: $V = \{1, 2, 3, 4, 5, 6\}$. Arcs and costs: $(1, 2)$: 4, $(1, 3)$: 2, $(2, 3)$: 1, $(2, 4)$: 5, $(3, 4)$: 8, $(3, 5)$: 10, $(4, 6)$: 2, $(5, 4)$: 1, $(5, 6)$: 6. Write the full LP and identify the net demands.

5. Transportation and Assignment Problems

5.1 Transportation Problem

A **transportation problem** is a special MCNF where every node is either a pure source or a pure sink — there are no transshipment nodes. All flow goes directly from sources to sinks.

Standard form (Rardin Definition 10.39): Given supplies s_i , demands d_j , and unit costs c_{ij} :

$$\begin{aligned} \min \quad & \sum_i \sum_j c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_j x_{ij} = s_i \quad \forall i \quad (\text{supply constraints}) \end{aligned}$$

$$\sum_i x_{ij} = d_j \quad \forall j \quad (\text{demand constraints})$$

$$x_{ij} \geq 0 \quad \forall i, j$$

Example (Rardin Example 10.22): Two sources (node 1: supply 50, node 3: supply 30) and two sinks (node 2: demand 45, node 4: demand 35) with costs $c_{1,2} = 7$, $c_{1,4} = 5$, $c_{3,2} = 4$, $c_{3,4} = 9$:

$$\begin{aligned} \min \quad & 7x_{1,2} + 5x_{1,4} + 4x_{3,2} + 9x_{3,4} \\ \text{s.t.} \quad & x_{1,2} + x_{1,4} = 50, \quad x_{3,2} + x_{3,4} = 30 \\ & x_{1,2} + x_{3,2} = 45, \quad x_{1,4} + x_{3,4} = 35; \quad x_{ij} \geq 0 \end{aligned}$$

5.2 Assignment Problem

Assignment problems are a further specialization where all supplies and demands equal 1, and decision variables are binary:

$$x_{ij} = \begin{cases} 1 & \text{if } i \text{ is assigned to } j \\ 0 & \text{otherwise} \end{cases}$$

Linear assignment formulation (Rardin Definition 10.40):

$$\begin{aligned} \min \quad & (\text{or max}) \quad \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{j:(i,j) \in A} x_{ij} = 1 \quad \forall i \quad (\text{every } i \text{ is assigned}) \\ & \sum_{i:(i,j) \in A} x_{ij} = 1 \quad \forall j \quad (\text{every } j \text{ is assigned}) \\ & x_{ij} \geq 0 \quad \forall (i,j) \in A \end{aligned}$$

Key insight: Even though assignment is naturally an integer (0-1) problem, the integrality property of network flows means we can solve it as an LP and automatically get an integer optimal solution.

Unequal sets: If one set is larger, the smaller set is augmented with **dummy** members connected to all members of the other set at zero cost.

5.3 Example: Dating Service Assignment (Rardin Example 10.23)

Three males and three females with compatibility ratings:

	$j = 1$	$j = 2$	$j = 3$
$i = 1$	90	30	12
$i = 2$	40	80	75
$i = 3$	60	65	80

Formulation (maximizing total compatibility):

$$\max 90x_{1,1} + 30x_{1,2} + 12x_{1,3} + 40x_{2,1} + 80x_{2,2} + 75x_{2,3} + 60x_{3,1} + 65x_{3,2} + 80x_{3,3}$$

subject to assignment constraints (each person paired exactly once) and $x_{ij} \geq 0$.

5.4 Application: CAM Workstation Assignment (Rardin Application 10.4)

Setting: A computer-aided manufacturing system routes 8 jobs to 10 workstations. Not every job can go to every station. Table entries show combined transportation + waiting + processing time for feasible job–station pairs (dashes indicate infeasible assignments).

Since there are 10 stations but only 8 jobs, 2 **dummy jobs** (9 and 10) are added with zero cost to all stations, making it a balanced 10×10 assignment. The optimal solution assigns:

$$x_{1,1}^* = x_{2,2}^* = x_{3,8}^* = x_{4,6}^* = x_{5,4}^* = x_{6,9}^* = x_{7,10}^* = x_{8,3}^* = 1$$

Exercise 5. A logistics company has 4 delivery trucks and 4 delivery zones. The cost (in \$) of assigning each truck to each zone is:

	Zone A	Zone B	Zone C	Zone D
Truck 1	90	75	110	95
Truck 2	35	85	55	65
Truck 3	125	45	80	105
Truck 4	50	90	60	70

- (a) Formulate as a minimization assignment problem.
- (b) If Truck 2 cannot serve Zone A, how does the formulation change?
- (c) If a 5th zone is added with costs $\{80, 40, 95, 55\}$, what structural change is needed?

6. Integrality Property of Network Flows (*Optional*)

6.1 The Key Result

One of the most powerful properties of network flow models:

Principle (Rardin 10.35): If a minimum cost network flow model has **integer constraint data** (supplies, demands, and capacities) and has any optimal solution, then it has an **integer optimal solution**.

This means:

- Transportation problems with integer data yield integer optimal flows
- Assignment problems (all supplies/demands = 1) yield 0/1 optimal solutions
- We can solve these as LPs and get integer solutions automatically — no branch-and-bound needed!

6.2 Why It Works

The property follows from two observations:

1. **Total unimodularity:** Node-arc incidence matrices are **totally unimodular** — every square submatrix has determinant $\in \{0, +1, -1\}$
2. **Basis computations:** When solving the LP, extreme-point (basic) solutions involve inverting submatrices of A . Under total unimodularity, if the right-hand side $\mathbf{b}$ and bounds $\mathbf{u}$ are integer, so is every basic feasible solution

Note: **Costs do not need to be integer.** The integrality property depends only on constraint data.

6.3 What This Does NOT Cover

Generalizations like **multicommodity flows** (multiple commodities sharing the same arcs) and **gain/loss flows** (where flow is multiplied along arcs) do **not** preserve total unimodularity. These problems may require integer programming techniques.

7. Matching Problems

7.1 Matching vs. Assignment

While assignment problems pair objects from two **distinct** sets, **matching problems** pair objects from a **single** set.

Decision variables:

$$x_{i,i'} = \begin{cases} 1 & \text{if } i \text{ is paired with } i' \\ 0 & \text{otherwise} \end{cases}$$

where by convention $i' > i$ to avoid double counting.

7.2 Matching Formulation (Rardin Definition 11.18)

$$\begin{aligned} & \min \text{ (or max) } \sum_i \sum_{i' > i} c_{i,i'} x_{i,i'} \\ \text{s.t. } & \sum_{i' < i} x_{i',i} + \sum_{i' > i} x_{i,i'} = 1 \quad \forall i \\ & x_{i,i'} \in \{0, 1\} \quad \forall i, i' > i \end{aligned}$$

The two sums in each constraint are needed because i could be the smaller or larger index in any pair.

7.3 Application: Superfi Speaker Matching (Rardin Application 11.7)

Setting: High-fidelity speaker manufacturer Superfi sells speakers in pairs. Even with strict quality standards, any two speakers produce some distortion $d_{i,i'}$ when paired. The goal is to pair all speakers in the current lot to minimize total distortion.

Formulation:

$$\begin{aligned} \min \quad & \sum_i \sum_{i' > i} d_{i,i'} x_{i,i'} \\ \text{s.t.} \quad & \sum_{i' < i} x_{i',i} + \sum_{i' > i} x_{i,i'} = 1 \quad \forall i \\ & x_{i,i'} \in \{0, 1\} \quad \forall i, i' \end{aligned}$$

7.4 Example: Team Formation Matching (Rardin Example 11.13)

An instructor assigns students to 2-person project teams. Each student s has rated their preference $p_{s,s'}$ for working with each other student. The model maximizes total two-way preference:

$$\max \sum_i \sum_{i' > i} (p_{i,i'} + p_{i',i}) x_{i,i'}$$

subject to matching constraints.

7.5 Matching Variants

Variant	Description
Maximum weight matching	Find a set of disjoint edges of maximum total weight
Perfect matching	Every node must be matched (all constraints are $= 1$)
Maximum cardinality matching	Match as many nodes as possible (relax $= 1$ to ≤ 1 , maximize count)
Bipartite matching	Matching on a bipartite graph (reduces to assignment)

Tractability note: Matching on bipartite graphs is polynomial (it reduces to assignment/network flow). Matching on general graphs is also polynomial but requires more sophisticated algorithms (Edmonds' blossom algorithm). However, unlike assignment, the matching polytope for general graphs is **not** described by the simple LP relaxation — additional constraints (e.g., odd-set constraints) are needed.

Exercise 6. Six tennis players need to be paired for 3 doubles practice matches. Their compatibility scores (combining skill balance + scheduling preference) are:

	P2	P3	P4	P5	P6
P1	8	5	9	3	7
P2	—	6	4	8	2
P3	—	—	7	5	9
P4	—	—	—	6	3
P5	—	—	—	—	4

- (a) Formulate as a maximum weight perfect matching problem.
- (b) Write out all the constraints explicitly.
- (c) How does the formulation change if Player 1 and Player 4 refuse to be paired?

8. Summary: The Network Flow Family

8.1 Special Cases of MCNF

The first five problems below form a hierarchy of progressively more specialized MCNF models. Each is obtained by restricting the parameters of the general MCNF formulation:

Problem	Special Case of MCNF?	How It Specializes MCNF
Min Cost Flow	— (this is the general model)	—
Max Flow	Yes	All $c_{ij} = 0$; all $b_k = 0$; add return arc (t, s) with $c_{t,s} = -1$
Shortest Path	Yes	Unit flow: $b_s = -1$, $b_t = +1$, all others $= 0$; uncapacitated
Transportation	Yes	Bipartite graph; no transshipment nodes; all arcs go source $\rightarrow$ sink
Assignment	Yes	Transportation with all $s_i = 1$, $d_j = 1$; binary optimal by integrality
Bipartite Matching	Yes	Equivalent to assignment on a bipartite graph
General Matching	No	Non-bipartite; constraint matrix is not a node–arc incidence matrix

8.2 Why General Matching Is Different

General (non-bipartite) matching problems pair objects from a **single** set, so the constraint matrix does not have the $\{+1, -1, 0\}$ column structure of a node–arc incidence matrix. As a result:

- The constraint matrix is **not** totally unimodular
- The LP relaxation can produce fractional solutions (e.g., $x = 1/2$ on every edge of a triangle satisfies all relaxed constraints)
- Solving to integrality requires either ILP techniques or specialized algorithms (e.g., Edmonds' blossom algorithm with odd-set inequalities)

Bipartite matching, by contrast, maps directly to an assignment problem on a bipartite graph and inherits all the MCNF properties, including guaranteed integer LP solutions.

8.3 Key Takeaways

1. The MCNF is a **linear program** whose node–arc incidence matrix is **totally unimodular**, guaranteeing integer optimal solutions when constraint data is integer.
 2. Recognizing that a problem has MCNF structure can transform an apparently difficult ILP into a tractable LP — this applies to max flow, shortest path, transportation, assignment, and bipartite matching.
 3. General matching is a closely **related** but **distinct** class of combinatorial optimization problems that requires tools beyond network flow LP.
-

9. What's Next

- **Computational lab:** Solving network flow problems in Python using AMPL/Gurobi on Google Colab
 - **Specialized algorithms:** Network simplex, cycle cancelling, Dijkstra, Bellman–Ford
 - **Advanced topics:** Multicommodity flows, flows with gains/losses, minimum spanning trees
-

References

- Rardin, R. L. (2017). *Optimization in Operations Research*, 2nd ed. Pearson. Chapters 9–11.
- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- Ford, L. R. & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399–404.